IDS update problem · New attacks with few labels

Intrusion detection system (IDS) · Classifies network records as benign or attack types

Labels available at deployment are insufficient for attack types observed later

Changing inputs

Variation in networks, services, and attacks
Traffic differs from the training period

Few attack labels

Labelled examples after investigation
Limited data for model updating

False alarms and update cost

Incorrect alarms on benign traffic
Additional computation for each update

Evaluate quality and cost after incorporating new labels, beyond one fixed closed-set score

Sommer · Paxson, Outside the Closed World: On Using Machine Learning for Network Intrusion Detection, IEEE S&P 2010 · Data scarcity, input variability, and error costs

Stage-wise detector updates based on the attack collection order

CSE-CIC-IDS2018 · Benign activity and scheduled attacks collected in an AWS testbed

CSE · UNB CIC, CSE-CIC-IDS2018 on AWS, 2018 · Official attack collection schedule

2/14	➡	2/15·16	➡	2/20·21	➡	2/22·23	➡	2/28·3/1	➡	3/2
Brute force		DoS		DDoS		Web attacks		Infiltration		Bot

Follow the original order of attack types, then select n labelled rows per attack from the same training split for each update

Compare · Post-update detection quality / retention of known classes / update time

Order retained; exact intervals and the complete traffic timeline are not replayed · ToN-IoT evaluated independently

TabPFN · A pretrained Transformer for tabular data

TabPFN · Transformer predictor pretrained on synthetic tabular tasks

Model used: v3 · Grinsztajn et al., TabPFN-3: Technical Report, arXiv 2026



In-context learning Predict a query from labelled examples without additional weight optimization

Working hypothesis Maintain detection quality with fewer new examples and lower update cost than retraining

This comparison: no IDS fine-tuning · context preprocessing and cache preparation are timed

Evaluation workflow · Updating a detector with few labels

① Label release and retention · CIC2018 example n = 5 · 10 · 20 · 30 · 50 · 100 per new attack · 3 seeds

Initial · Stage 1 · 1,000 rows

Benign 750	Brute force 125	DoS 125

New · Stage 2 DDoS n rows → **New · Stage 3** Web attacks n rows → **New · Stage 4** Infiltration n rows → **New · Stage 5** Bot n rows

ToN uses its own training split and attack stages · allocations in the appendix

Accum. Final 1,000+4n

Retain every labelled row released so far

Buffer 1,000 rows

Evict old rows → insert all n new rows

Eviction · FIFO / random / similarity

② Update at every stage

XGB refit model

TabPFN refresh context and cache

Pretrained weights frozen

③ Evaluate on the same test set

Final model · full test set

CIC	3,042,473 rows
ToN	2,271,723 rows

Separate per benchmark

④ Outcomes

Detection quality

Full test · overall and new-attack F1

Retention of known classes

Fixed panel · final – initial F1

Update cost

Sum after Stage 1 · inference excluded